

Bài 1 :

1. student.name có bị đổi không?

Không đổi, vẫn là 'hoang'.

2. student.parent.name có bị đổi không?

Có bị đổi thành 'bo bang'.

3. Giải thích vì sao?

Vấn đề nằm ở chỗ toán tử ... (spread operator) chỉ thực hiện **sao chép nông (shallow copy)** thôi:

- **Với thuộc tính ở lớp ngoài (name):** Vì nó là kiểu dữ liệu nguyên thủy (string), nên khi dùng ..., JS sẽ copy giá trị sang một vùng nhớ mới hoàn toàn. mentor.name thay đổi thì không liên quan gì đến student.name.
- **Với thuộc tính lồng bên trong (parent):** Đây là một object. Lúc này, thằng ... không tạo ra object parent mới mà nó chỉ copy cái **địa chỉ (reference)** của object đó thôi.

Kết quả là cả student.parent và mentor.parent đều đang trỏ chung vào một dữ liệu trong bộ nhớ. Thế nên khi sửa tên parent qua biến mentor thì parent bên biến student cũng bị đổi theo luôn vì thực chất cả hai là một.

Bài 2 :

1. student.parent.name có bị ảnh hưởng không?

Giá trị của nó vẫn là 'bo hoang'.

2. Vì sao cách này khác với dùng Spread (...)?

Cách dùng JSON.parse(JSON.stringify(student)) tạo ra một **Deep Copy** (sao chép sâu), còn toán tử Spread chỉ là **Shallow Copy** (sao chép nông).

Giải thích chi tiết:

- **Cơ chế của cách này:** Khi dùng JSON.stringify, toàn bộ object (bao gồm cả các object lồng bên trong như parent) sẽ bị biến thành một chuỗi văn bản (string). Sau đó, JSON.parse sẽ cầm chuỗi đó để xây dựng lại một object hoàn toàn mới từ đầu.

- **Kết quả là:** Object mentor lúc này không còn bất kỳ sợi dây liên kết nào về bộ nhớ với object student ban đầu nữa. Mỗi thuộc tính, dù là ở cấp độ nào, đều nằm ở các ô nhớ riêng biệt.
- **Khác với Spread:** Ở bài trước, Spread chỉ copy cái vỏ bên ngoài và giữ nguyên địa chỉ của các object con bên trong. Còn cách này thì "đập đi xây lại" toàn bộ, nên khi bạn sửa tên parent ở mentor, parent ở student hoàn toàn không hề hay biết gì.

Bài 3 :

1. Mảng có bị thay đổi không?

Về mặt nội dung, mảng students **có bị thay đổi**. Mặc dù newStudents là một mảng mới, nhưng dữ liệu bên trong của nó và mảng gốc đang bị dính chùm với nhau.

2. Phần tử bên trong có bị không?

Shallow Copy (sao chép nông) quen thuộc, nhưng lần này là áp dụng cho mảng:

- Khi bạn dùng Spread [...], JavaScript tạo ra một cái mảng mới hoàn toàn cho newStudents.
- Tuy nhiên, các object { name: 'a' } đặt trên cái mảng mới đó thì JS không hề nhân bản ra cái mới. Nó chỉ copy cái **địa chỉ** của những object đó từ mảng cũ sang mảng mới thôi.

Vì cả students[0] và newStudents[0] đều đang trỏ chung vào cùng một object trong bộ nhớ, nên khi sửa name = 'z' thông qua mảng newStudents, thì khi nhìn lại mảng students, bạn sẽ thấy object của nó cũng đã bị sửa y hệt.

Bài 4

Kết quả là: 999

Giải thích :

Vẫn là do cơ chế **Shallow Copy (sao chép nông)** của toán tử Spread (...), nhưng ở bài này mức độ lồng nhau sâu hơn một chút (3 cấp):

- Khi dùng `const newUser = { ...user }`, JavaScript chỉ thực hiện nhân bản các giá trị ở **cấp độ 1** (là name và cái "địa chỉ" của object address).
- Đối với các object lồng sâu bên trong như address và location, toán tử Spread **không** đi sâu vào bên trong để copy chúng. Nó chỉ đơn giản là bê nguyên cái địa chỉ vùng nhớ từ user sang newUser.

Vì vậy, thực tế trong bộ nhớ:

1. `user.address` và `newUser.address` là **một**.
2. `user.address.location` và `newUser.address.location` cũng là **một**.

Khi thay đổi lat ở bất cứ cấp độ sâu nào thông qua biến `newUser`, bạn thực chất đang sửa đổi trực tiếp trên đối tượng dùng chung đó. Do đó, khi in `user.address.location.lat`, kết quả sẽ là giá trị mới đã bị ghi đè.